



UNIOESTE - Universidade Estadual do Oeste do Paraná  
Centro de Ciências Exatas e Tecnológicas - CCET  
Curso de Ciência da Computação

Pedro Lucas Moraes, Thalita Wiederkehr Pereira e Vinícius Mattos Marcos

## Especificação do Sistema de Arquivos CBFS

## 1. Introdução

Este documento detalha a especificação do sistema de arquivos **CBFS (Contiguous Bitmap File System)**, que combina a técnica de **Alocação Contígua** para gerenciamento de arquivos e o **Bitmap** para gerenciamento de espaço livre. O objetivo é fornecer todos os detalhes necessários para que seja possível:

- Interpretar a estrutura do sistema de arquivos, ou seja, localizar arquivos, determinar seus tamanhos e mapear a alocação de seus blocos de dados.

## 2. Estrutura do Volume

O sistema de arquivos CBFS é organizado em setores de 512 bytes, que é a menor unidade de alocação de dados. O volume é dividido nas seguintes regiões (figura 1), em ordem sequencial:

1. **Setor de Boot (Boot Record)**: Contém os parâmetros de geometria do disco e informações de inicialização. Ocupando sempre o primeiro setor do disco.
2. **Bitmap de Espaço Livre**: Uma região que armazena o estado de alocação de cada setor na Região de Dados com apenas um nível.
3. **Diretório Raiz (Root Directory)**: Contém as entradas de metadados para os arquivos e subdiretórios.
4. **Região de Dados (Data Region)**: Contém o conteúdo real dos arquivos.

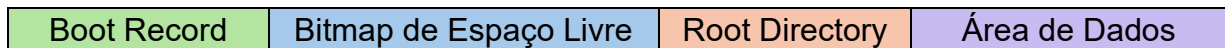


Figura 1: Disco dividido por setores.

## 3. Estruturas de Dados

Todos os valores numéricos maiores que 1 byte (UINT16, UINT32) devem ser armazenados no formato **Little-Endian**.

### 3.1. Setor de Boot (*Boot Record*)

O Setor de Boot ocupa o primeiro setor do volume (Setor 0).

| Offset<br>(Bytes) | Tamanho<br>(Bytes) | Tipo     | Descrição                   | Valor/Fórmula                                                  |
|-------------------|--------------------|----------|-----------------------------|----------------------------------------------------------------|
| 0                 | 2                  | UINT16   | Bytes por Setor             | 512                                                            |
| 2                 | 2                  | UINT16   | Setores Reservados          | 1 (Apenas o boot)                                              |
| 4                 | 2                  | UINT16   | Máx. Entradas no Root       | Padrão: 128                                                    |
| 6                 | 2                  | UINT16   | Tam. do Root (Setores)      | $8 \text{ setores} = \left( \frac{128 \times 32}{512} \right)$ |
| 8                 | 4                  | UINT32   | Total de Setores do Disco   | Definido na formatação (N)                                     |
| 12                | 4                  | UINT32   | Início do Bitmap (Setor)    | 1 (Logo após Boot)                                             |
| 16                | 4                  | UINT32   | Tamanho do Bitmap (Setores) | $\left\lceil \frac{N}{4096} \right\rceil$                      |
| 20                | 4                  | UINT32   | Início do Root Dir (Setor)  | 1 + Tam. Bitmap                                                |
| 24                | 4                  | UINT32   | Início da Área de Dados     | Início Root + Tam. Root                                        |
| 28                | 4                  | PAD      | Reservado (Padding)         | Preenchido com zeros                                           |
| 32                | 4                  | CHAR [4] | Nome do Sistema             | "CBFS"                                                         |
| 36                | 2                  | UINT16   | Assinatura de Boot          | 0x7777                                                         |
| 38                | 474                | PAD      | Padding Final               | Preenchido com zeros                                           |

Tabela 1: Boot Record.

**Nota:** O tamanho do Bitmap é calculado dividindo o total de setores por 4096 (capacidade de mapeamento de 1 setor: 512 x 8 bits). O resultado deve ser arredondado para cima para garantir que todos os setores do disco sejam mapeados.

3.2. Bitmap de Espaço Livre

Localizado imediatamente após o Boot Record (Setor 1). Mapeias o estado de cada setor do disco (inclusive os de metadados).

- **Mapeamento:** 1 Bit no Bitmap = 1 Setor físico no disco.
- **Estados:**
  - 0: Setor Livre
  - 1: Setor Ocupado.
- **Inicialização:** Os setores ocupados pelos metadados (Boot, Bitmap e Root) devem ser marcados como 1 (Ocupado) no momento da formatação para evitar sobrescrita.

3.3. Diretório Raiz (*Root Directory*)

Localizado após o Bitmap. O Diretório Raiz armazena as entradas de metadados dos arquivos.

- **Estrutura:** Diretório Plano (*Flat Directory*). Não há suporte para subpastas.
- **Capacidade:** Padrão de 128 arquivos.
- **Entrada:** Tamanho fixo de 32 bytes.

| Estrutura da Entrada (32 bytes) |                 |        |                                                                             |
|---------------------------------|-----------------|--------|-----------------------------------------------------------------------------|
| Offset (Bytes)                  | Tamanho (Bytes) | Tipo   | Descrição                                                                   |
| 0                               | 16              | ASCII  | <b>Nome do Arquivo:</b> Alinhado à esquerda, preenchido com espaços (0x20). |
| 16                              | 3               | ASCII  | <b>Extensão:</b> Ex: "TXT", "JPG".                                          |
| 19                              | 1               | UINT8  | <b>Atributos:</b> Conforme a tabela de atributos.                           |
| 20                              | 4               | UINT32 | <b>Tamanho em Bytes:</b> Tamanho real do arquivo.                           |
| 24                              | 4               | UINT32 | <b>Primeiro Setor:</b> Número do Setor Físico (LBA) onde o arquivo começa.  |
| 28                              | 4               | UINT32 | <b>Número de Setores:</b> Quantidade de setores contíguos alocados.         |

Tabela 2: Root Directory.

### 3.4. Tabela de Atributos

O byte de atributo (Offset 0x13 da entrada) define o status do arquivo.

| Tabela de Atributos (Offset 19) |                    |                                                                                                      |                                                     |
|---------------------------------|--------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Valor (Hex)                     | Estado             | Descrição Técnica                                                                                    | Ação na Leitura                                     |
| 0x00                            | Livre (Fim)        | Entrada nunca utilizada e nenhuma entrada posterior existe.                                          | <b>Parar</b> a leitura imediatamente (Otimização).  |
| 0x10                            | Arquivo Válido     | O arquivo está gravado, íntegro e nunca foi alterado após a criação.                                 | <b>Ler/Listar</b> o arquivo.                        |
| 0x20                            | Excluído           | O arquivo foi apagado. O espaço no Bitmap é livre, mas o nome persiste.                              | <b>Ignorar</b> e pular para a próxima entrada.      |
| 0x30                            | Arquivo Modificado | O arquivo é válido, mas seu conteúdo foi alterado após a criação inicial. (Lógica de "Archive Bit"). | <b>Ler/Listar</b> o arquivo (Tratar igual ao 0x10). |

Tabela 3: Tabela de Atributos.

### 3.5. Área de Dados (*Data Region*)

A Região de Dados armazena o conteúdo dos arquivos, sendo a região final do disco. A alocação é estritamente **contígua**.

- **Estrutura:** Os dados são armazenados em setores, cada setor é obrigatoriamente ocupado por apenas um arquivo mesmo que ocupe poucos bytes.
- Se não houver um bloco contíguo livre o suficiente para acomodar o arquivo, a operação de escrita falhará, mesmo que a soma dos espaços livres fragmentados seja suficiente.

## 4. Manual de uso

Para utilizar o sistema de arquivos é necessário primeiramente compilar o código com: `g++ CBFS.cpp -o cbfs`.

Este sistema opera completamente por linha de comando, seguindo o padrão:

```
./cbfs <comando> <disco> [argumentos]
```

Onde:

<comando> define a operação a ser realizada;

<disco> é o arquivo que representa o disco do sistema CBFS;

[argumentos] são parâmetros adicionais, quando necessários.

#### 4.1. Formatar disco

Formata um arquivo .img com as estruturas de dados necessárias do sistema de arquivos.

- **Sintaxe:** `./cbfs formatar <nome_do_disco> <total_de_setores>`
- **Exemplo:** `./cbfs formatar pendrive.img 2048`

Notas:

- Cada setor tem 512 bytes.
- No exemplo acima, o disco terá aproximadamente **1 MB** ( $2048 \times 512$  bytes).
- Qualquer conteúdo anterior do arquivo será sobrescrito.

#### 4.2. Listar arquivos

Exibe todos os arquivos armazenados no diretório raiz do CBFS, apresentando: nome do arquivo, extensão, seu tamanho em byte, e o setor inicial de alocação (LBA).

- **Sintaxe:** `./cbfs listar <nome_do_disco>`
- **Exemplo:** `./cbfs listar pendrive.img`

#### 4.3. Importar arquivo (PC para o Sistema de Arquivos)

Copia um arquivo do seu computador para dentro do sistema CBFS.

- **Sintaxe:** `./cbfs importar <nome_do_disco> <arquivo_origem>`
- **Exemplo:** `./cbfs importar pendrive.img CBFS.cpp`

Notas:

- O nome do arquivo no CBFS é composto por até 16 caracteres para o nome e 3 para a extensão.
- Nomes maiores são truncados automaticamente.
- Os nomes são convertidos automaticamente para letras maiúsculas.
- O arquivo é armazenado de forma contígua na região de dados.

#### 4.4. Exportar arquivo (Sistema de Arquivos para o PC)

Recupera/copia um arquivo de dentro do CBFS e o salva no seu computador no diretório atual.

- **Sintaxe:** `./cbfs exportar <nome_do_disco>  
<nome_do_arquivo_no_cbfs>`
- **Exemplo:** `./cbfs exportar pendrive.img CBFS.cpp`

Notas:

- O nome do arquivo deve ser informado exatamente como armazenado no CBFS (normalmente em letras maiúsculas).
- O conteúdo do arquivo é reconstruído a partir dos setores alocados.

## 4.5. Excluir Arquivo

Exclui um arquivo do sistema, liberando os bits no bitmap e os blocos na região de índices.

- **Sintaxe:** `./cbfs remover <nome_do_disco>  
<nome_do_arquivo_no_cbfs>`
- **Exemplo:** `./cbfs remover pendrive.img arquivo.txt`

*Notas:*

- *A exclusão é lógica; os dados permanecem no disco até serem sobrescritos.*
- *Os setores liberados podem ser reutilizados por novas importações.*

## 4.6. Verificar Status

Exibe um mapa visual da ocupação do disco e estatísticas de uso, permitindo diagnosticar a fragmentação do volume.

- **Sintaxe:** `./cbfs status <nome_do_disco>`
- **Exemplo:** `./cbfs status pendrive.img`
- **Saída:** A saída apresenta um gráfico onde # representa setores ocupados e . representa setores livres, além do cálculo percentual de uso.